MACHINE LEARNING

A to Z — Complete Reference Guide

From Data Preprocessing to Deep Learning & Beyond

Covers: Data Preprocessing • Regression • Classification • Clustering
Association Rules • Reinforcement Learning • NLP • Deep Learning • Dimensionality Reduction

Production-Ready ML Engineering

Chapter 1: Data Preprocessing

Data preprocessing is the foundational step in every machine learning pipeline. Raw data collected from real-world sources is almost never in a clean, analysis-ready format. It may contain missing values, inconsistent data types, categorical text labels, or features at vastly different numerical scales. Preprocessing transforms this raw data into a structured, numerical, and normalized form that machine learning algorithms can effectively learn from.

Think of preprocessing as preparing ingredients before cooking. No matter how skilled the chef or how sophisticated the recipe, poor-quality ingredients will ruin the dish. Similarly, even the most powerful ML algorithm will underperform on poorly prepared data.

SECTION 1 **Theory — Understanding Data Preprocessing**

1.1 The Machine Learning Workflow

Before diving into preprocessing specifics, it is important to understand where preprocessing sits in the overall ML workflow. The three main stages are:

1. Data Preprocessing — Clean, encode, scale, and split the raw data.
2. Model Building — Select and train a suitable ML algorithm on the training data.
3. Model Evaluation — Assess the trained model's performance on unseen test data.

Every subsequent chapter in this book follows these same three stages. Preprocessing is always the first — and arguably most critical — step.

1.2 Training Set vs. Test Set

One of the first decisions in preprocessing is splitting your dataset into two distinct parts: a training set and a test set.

Concept	Explanation
Training Set	The portion of data used to train (fit) the ML model. Typically 80% of the dataset.
Test Set	The portion kept completely hidden from the model during training. Used only to evaluate performance. Typically 20%.
Split Ratio	Standard practice is 80:20, though this can vary based on dataset size.
Why Split?	To simulate real-world performance — the model must predict on data it has never seen before.

Example: Suppose you want to predict car sale prices. You train your model on 800 car records (with known prices) and then test it on 200 new car records. If the model predicts prices accurately on those 200 unseen records, you can trust it will perform well in production.

1.3 Taking Care of Missing Values

Real-world datasets almost always have missing values — entries where data was not recorded. Missing values cause most ML algorithms to throw errors or produce incorrect results.

There are two main strategies for handling missing values:

- Strategy 1 — Deletion: Simply remove the rows (observations) that contain missing values. This is fast but risky — if many rows have missing values, or if the missing data is not random, you may introduce bias and lose valuable information.
- Strategy 2 — Imputation: Replace missing values with a statistical estimate computed from the existing data. The most common approach is replacing missing values with the mean (average) of the column. Other options include the median or the mode (most frequent value).

Why Imputation is Usually Preferred

Deleting rows reduces your dataset size, which is harmful especially when data is scarce. Imputation preserves the dataset size while filling in reasonable estimates. For numerical columns, mean imputation is the default. For categorical columns, mode imputation (replacing with the most frequent category) is used.

1.4 Encoding Categorical Data

Machine learning models are mathematical in nature — they work exclusively with numbers. However, real-world datasets frequently contain text-based categorical columns (e.g., Country: 'France', 'Germany', 'Spain' or Purchased: 'Yes', 'No'). These must be converted to numerical representations.

1.4.1 One-Hot Encoding (for Independent Variables)

One-Hot Encoding (OHE) converts a categorical column with N unique categories into N separate binary (0 or 1) columns. Each new column represents one category, and for each row, exactly one of those columns has the value 1 while the rest are 0.

Example: The Country column with values {France, Germany, Spain} becomes three binary columns:

Original Value	Encoded Representation
France	France=1, Germany=0, Spain=0
Germany	France=0, Germany=1, Spain=0

Spain	France=0, Germany=0, Spain=1

Why not just assign numbers like France=1, Germany=2, Spain=3? Because that would create an artificial ordinal relationship — the model might interpret Germany (2) as being 'greater than' France (1), which is meaningless for categorical data.

1.4.2 Label Encoding (for the Dependent Variable)

For the dependent variable (output), which is typically binary (Yes/No, True/False, 0/1), we use Label Encoding rather than One-Hot Encoding. Label Encoding maps each category to a simple integer:

- 'No' → 0
- 'Yes' → 1

Since the dependent variable is the output we are trying to predict, there is no risk of the model misinterpreting the numerical ordering. The model simply needs to distinguish between two categories.

1.5 Feature Scaling

Feature scaling ensures that all numerical features in the dataset are on the same scale. Without scaling, features with larger numerical ranges (e.g., Salary: 50,000–150,000) will numerically dominate features with smaller ranges (e.g., Age: 18–65), causing the model to give disproportionate weight to the larger-scaled feature.

1.5.1 Normalisation (Min-Max Scaling)

Normalisation rescales all values in a column to fit within the range [0, 1]. The formula is:

```
X_normalized = (X - X_min) / (X_max - X_min)
```

Normalisation is best suited when the data is known to follow a bounded distribution (i.e., you know the minimum and maximum possible values, and they do not vary significantly in new data).

1.5.2 Standardisation (Z-Score Scaling)

Standardisation transforms values so that they have a mean of 0 and a standard deviation of 1. The formula is:

```
X_standardized = (X - mean) / standard_deviation
```

The resulting values typically fall within the range [-3, +3] (covering 99.7% of data under a normal distribution). Standardisation works well across almost all scenarios, which is why it is the default choice in practice.

Normalisation	Standardisation
Range: [0, 1]	Range: typically [-3, +3]
Best when data is normally distributed and bounded	Works well always — the universal default
Sensitive to outliers (outliers compress the rest)	More robust to outliers
Use: Neural networks, image pixel scaling	Use: Linear models, SVM, KNN, clustering

Critical Rule: Apply Feature Scaling AFTER Splitting

Feature scaling must be applied AFTER splitting the data into training and test sets. This is because StandardScaler computes the mean and standard deviation from the data it is fitted on. If you scale before splitting, the scaler uses statistics computed from both training AND test data — this constitutes information leakage, where the model indirectly learns characteristics of the test set during training. Always fit the scaler on training data only, then apply (transform) the same scaler to the test data.

1.6 Splitting the Dataset into Training and Test Sets

The split is performed using scikit-learn's train_test_split function, which randomly shuffles and divides the data. Key parameters:

- test_size: The proportion of data allocated to the test set. E.g., 0.2 means 20%.
- random_state: A seed for the random number generator, ensuring reproducibility (running the code twice gives the same split).

SECTION 2 **Practical — Data Preprocessing Notebook**

Notebook: data_preprocessing.ipynb

The following notebook walks through a complete data preprocessing pipeline on a real dataset. Each cell is explained in detail.

Dataset Overview

We use a dataset called Data.csv containing customer information. It has the following columns:

Column	Description

Country	Categorical — Customer's country (France, Spain, Germany)
Age	Numerical — Customer's age (some missing values)
Salary	Numerical — Annual salary in USD (some missing values)
Purchased	Categorical — Target variable: Yes or No

Cell 1 — Importing the Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

What this cell does: This is the standard boilerplate for any ML project in Python. We import three essential libraries:

- numpy (np): Provides support for numerical arrays and matrix operations. Nearly all ML data structures are NumPy arrays under the hood.
- matplotlib.pyplot (plt): The primary plotting library. Used to visualize data distributions, model predictions, and evaluation results.
- pandas (pd): Provides the DataFrame data structure for loading, inspecting, and manipulating structured tabular data (like CSV files). Also used for feature extraction and transformation.

Cell 2 — Importing the Dataset

```
dataset = pd.read_csv('Data.csv')

# Create the feature matrix X (all rows, all columns except the last)
X = dataset.iloc[:, :-1].values

# Create the dependent variable vector y (all rows, last column only)
y = dataset.iloc[:, -1].values
```

What this cell does:

- pd.read_csv('Data.csv'): Reads the CSV file and loads it into a Pandas DataFrame — a 2D labeled table structure similar to an Excel spreadsheet.
- dataset.iloc[:, :-1].values: Uses iloc (Integer Location Indexer) to select all rows (:) and all columns except the last (:-1). The .values converts the DataFrame slice into a NumPy array, which is the format required by scikit-learn.
- dataset.iloc[:, -1].values: Selects all rows and only the last column — this is our target variable (Purchased). The result is a 1D NumPy array.

After this cell: X is a 2D NumPy array of shape (n_samples, n_features). y is a 1D NumPy array of shape (n_samples,).

Cell 3 — Taking Care of Missing Values

```
from sklearn.impute import SimpleImputer

# Create an imputer that replaces missing values (NaN) with the column mean
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')

# Fit the imputer on the numerical columns (columns 1 and 2: Age and Salary)
imputer.fit(X[:, 1:3])

# Apply the imputation (replace NaN values with computed means)
X[:, 1:3] = imputer.transform(X[:, 1:3])
```

What this cell does:

- SimpleImputer(missing_values=np.nan, strategy='mean'): Creates an imputer object. The strategy='mean' argument tells it to replace any NaN (missing) value with the arithmetic mean of that column.
- imputer.fit(X[:, 1:3]): The .fit() method computes the statistics (in this case, the mean of Age and mean of Salary) from the data. We only apply this to columns 1 and 2 (Age and Salary) because column 0 (Country) is categorical — imputing a mean for a text column would be meaningless.
- X[:, 1:3] = imputer.transform(X[:, 1:3]): The .transform() method applies the imputation, replacing all NaN entries with the pre-computed column means. We write the result back into X.

Why fit() and transform() separately?

The scikit-learn API separates 'learning' (fit) from 'applying' (transform). This matters enormously when dealing with training/test splits: you always fit on training data only, then transform both training and test data using the statistics learned from training. This prevents information leakage from test data.

Cell 4 — Encoding Categorical Data: Independent Variable

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

# Apply One-Hot Encoding to column 0 (Country)
# remainder='passthrough' keeps all other columns unchanged
ct = ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(), [0])],
    remainder='passthrough'
)

# fit_transform: fit the encoder AND apply transformation in one step
X = np.array(ct.fit_transform(X))
```

What this cell does:

- ColumnTransformer: A powerful transformer that applies different preprocessing steps to different columns simultaneously. Here, we apply OneHotEncoder only to column 0 (Country), while leaving the remaining columns (Age, Salary) unchanged (remainder='passthrough').

- OneHotEncoder(): Converts the Country column into three binary columns: one for France, one for Germany, one for Spain.
- ct.fit_transform(X): Simultaneously fits the encoder (learns the unique categories) and transforms the data. We wrap the result in np.array() to ensure the output is a standard NumPy array (ColumnTransformer can return a sparse matrix).

After this cell: X now has three additional binary columns (for France, Germany, Spain) at the beginning, and the original Country string column has been removed. The total number of columns has increased from 3 to 5.

Cell 5 — Encoding the Dependent Variable

```
from sklearn.preprocessing import LabelEncoder

# Encode the target variable: 'No' -> 0, 'Yes' -> 1
le = LabelEncoder()
y = le.fit_transform(y)
```

What this cell does:

- LabelEncoder(): Assigns a unique integer to each unique string value in y. It sorts the categories alphabetically and assigns integers: 'No' -> 0, 'Yes' -> 1.
- le.fit_transform(y): In one step, fits the encoder (learns the unique categories) and transforms y into a numerical array.

After this cell: y is now a NumPy array of 0s and 1s instead of 'No' and 'Yes' strings.

Cell 6 — Splitting the Dataset

```
from sklearn.model_selection import train_test_split

# Split X and y into training and test sets
# test_size=0.2  -> 20% goes to test, 80% stays for training
# random_state=1 -> ensures reproducible splits across runs
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

print('Training set size:', X_train.shape)
print('Test set size:    ', X_test.shape)
```

What this cell does: train_test_split randomly shuffles the dataset and divides it into four arrays:

- X_train: Feature matrix for training (80% of rows)
- X_test: Feature matrix for testing (20% of rows)
- y_train: Target vector for training
- y_test: Target vector for testing — the 'ground truth' we compare predictions against

The random_state=1 parameter seeds the random number generator, ensuring that every time you run this cell, you get the exact same split. This is essential for reproducibility in research and production.

Cell 7 — Feature Scaling

```
from sklearn.preprocessing import StandardScaler

# Create a StandardScaler instance
sc = StandardScaler()

# IMPORTANT: Fit ONLY on X_train, then transform both X_train and X_test
X_train[:, 3:] = sc.fit_transform(X_train[:, 3:])

# Use the SAME scaler (same mean & std) to transform X_test
X_test[:, 3:] = sc.transform(X_test[:, 3:])
```

What this cell does:

- StandardScaler(): Creates a scaler that will standardize features by removing the mean and scaling to unit variance.
- sc.fit_transform(X_train[:, 3:]): Fits the scaler on X_train — computing the mean and standard deviation for the Age and Salary columns — and simultaneously applies the transformation. We use [:, 3:] to skip the first three columns (the One-Hot Encoded dummy binary columns for France, Germany, Spain), which must NOT be scaled.
- sc.transform(X_test[:, 3:]): Applies the transformation to X_test using the mean and standard deviation computed from X_train. This is critical: we do NOT call fit_transform on X_test because we want to use the same scaling statistics from the training set, preventing information leakage.

Why skip the dummy variables during scaling?

The first three columns (0, 1, 2) are One-Hot Encoded binary columns containing only 0s and 1s. These already exist in a standardized range. Applying StandardScaler on them would change their clean binary nature and make them harder to interpret. Always skip dummy/binary columns during feature scaling.

Chapter 2: Regression

Regression is a supervised learning technique used to predict a continuous numerical output variable based on one or more input features. Unlike classification (which predicts categories), regression predicts quantities — such as house prices, temperatures, sales figures, or salaries.

In this chapter, we cover five regression algorithms, progressing from the simplest (Simple Linear Regression) to more powerful non-linear approaches (SVR, Decision Trees, Random Forests), along with model evaluation using R-squared.

SECTION 1 Theory — Regression Algorithms

2.1 Simple Linear Regression

Simple Linear Regression models the relationship between one independent variable (feature) and one continuous dependent variable (output) using a straight line. The mathematical equation is:

```
y = b0 + b1 * x1
```

- y: The dependent variable (what we want to predict)
- b0: The y-intercept (the value of y when x=0)
- b1: The slope coefficient (how much y changes for a one-unit increase in x)
- x1: The independent variable

The best-fit line is found using the Ordinary Least Squares (OLS) method, which minimizes the sum of the squared differences between the actual values and the predicted values (residuals).

2.2 Multiple Linear Regression

Multiple Linear Regression extends Simple Linear Regression to handle multiple independent variables simultaneously:

```
y = b0 + b1*x1 + b2*x2 + ... + bn*xn
```

Each feature xi has its own slope coefficient bi, representing its individual contribution to the prediction.

5 Assumptions of Linear Regression

Before applying linear regression, verify that your data satisfies these assumptions:

4. Linearity: There must be a linear relationship between Y and each X variable.

5. Homoscedasticity: The variance of the residuals (errors) must be constant across all levels of the independent variables.
6. Multivariate Normality: The residuals should follow a normal distribution.
7. Independence: Observations should be independent of each other (no autocorrelation — no sequential patterns in the data).
8. No Multicollinearity: The independent variables should not be highly correlated with each other. If they are, the coefficient estimates become unstable and unreliable.

Dummy Variable Trap

When using categorical variables encoded as dummy variables (e.g., France=1/0, Germany=1/0, Spain=1/0), you must always omit one dummy variable from the model. If you include all three, perfect multicollinearity results (France + Germany + Spain = 1 for every row). Always use N-1 dummy variables for N categories. scikit-learn's OneHotEncoder with drop='first' handles this automatically.

P-Values and Statistical Significance

The p-value of each feature tells you whether that feature has a statistically significant effect on the output variable:

p-value Threshold	Interpretation
p < 0.05	Reject the null hypothesis. The feature has a statistically significant effect. Keep it in the model.
p >= 0.05	Fail to reject the null hypothesis. The feature may have no meaningful effect. Consider removing it.
p = 0.01	Very strong evidence of significance (1% significance level)

Model Building Strategies

Not all features should be included in your model. Two main reasons to be selective:

- Garbage In, Garbage Out: Irrelevant features add noise, degrading model performance.
- Interpretability: A simpler model with fewer features is easier to explain and debug.

Five approaches to select features:

9. All-in: Use all available variables. Best when you have prior domain knowledge that all are relevant.
10. Backward Elimination: Start with all variables. At each step, remove the variable with the highest p-value (above 0.05). Repeat until all remaining variables have p < 0.05.
11. Forward Selection: Start with no variables. At each step, add the variable that most improves the model (lowest p-value). Stop when no additional variable is significant.

12. Bidirectional Elimination (Stepwise): A combination of forward selection and backward elimination. At each step, both add and remove variables based on significance criteria.
13. All Possible Models: Test every possible combination of variables and select the best-scoring one. Computationally expensive and rarely used in practice.

2.3 Polynomial Regression

Polynomial Regression captures non-linear relationships between a single independent variable and the dependent variable by adding polynomial (power) terms:

```
y = b0 + b1*x + b2*x^2 + b3*x^3 + ... + bn*x^n
```

Despite having non-linear x terms, this is still called a 'linear' model because 'linear' in statistical modeling refers to linearity in the coefficients (b0, b1, b2, ...), not in the features. The coefficients are still estimated using linear algebra.

2.4 Support Vector Regression (SVR)

SVR takes a fundamentally different approach from ordinary linear regression. Instead of minimizing the sum of squared errors for all data points, SVR:

- Defines an epsilon-insensitive tube around the regression line. Data points inside the tube contribute ZERO error — SVR does not try to fit them more precisely.
- Only penalizes data points that fall OUTSIDE the tube. The distance from these outlier points to the tube boundary determines the model's loss.
- The data points outside the tube (that influence the tube's shape) are called support vectors.

SVR with a non-linear kernel (e.g., RBF — Radial Basis Function) can model complex, non-linear relationships. Unlike most regression models, SVR requires explicit feature scaling (StandardScaler) on both X and y.

2.5 Decision Tree Regression

Decision Tree Regression is a non-parametric algorithm that makes predictions by splitting the feature space into rectangular regions and assigning the average target value of training points in each region.

It works by recursively splitting the dataset based on the feature and threshold that minimizes the variance within each resulting subset (measured using metrics like information gain). The result is an interpretable tree of if-else rules.

Decision trees do NOT require feature scaling because they make splits based on thresholds, not distances or gradient computations.

2.6 Random Forest Regression

Random Forest is an ensemble learning algorithm that builds multiple decision trees and aggregates their predictions:

14. Select N random subsets (bootstrap samples) from the training data.
15. Build a Decision Tree on each bootstrap sample, using a random subset of features at each split.
16. For regression: take the average of all N trees' predictions as the final output.

Random Forests are significantly more robust and accurate than individual decision trees because:

- Averaging multiple trees reduces variance (overfitting).
- Each tree is built on a different random subset, so errors are uncorrelated across trees.

2.7 Evaluating Regression Models: R-Squared (R^2)

R^2 (R-squared) measures how well the regression model explains the variance in the dependent variable. It ranges from 0 to 1 (and can be negative for very poor models):

R^2 Value	Interpretation
1.0	Perfect fit — suspicious of overfitting
~0.9	Excellent fit
~0.7–0.9	Good fit
~0.5–0.7	Moderate fit — may need improvement
< 0.4	Poor fit — model explains little variance
< 0	Model is worse than a simple horizontal mean line

Adjusted R^2 penalizes the addition of irrelevant features that do not actually improve the model. Use Adjusted R^2 when comparing models with different numbers of features — it prevents the false impression that adding more variables always improves the model.

SECTION 2	Practical — Regression Notebooks

Notebook: simple_linear_regression.ipynb

Dataset: Salary_Data.csv — Years of Experience vs. Salary

Cell 1 — Data Preprocessing

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split

dataset = pd.read_csv('Salary_Data.csv')
X = dataset.iloc[:, :-1].values   # Years of Experience
y = dataset.iloc[:, -1].values    # Salary

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
```

What this cell does: Standard preprocessing — load the CSV, separate features (X) from target (y), and split 80/20 into training and test sets. Note: For Simple Linear Regression, we do not apply feature scaling because LinearRegression from scikit-learn internally handles the scaling during the OLS computation.

Cell 2 — Training the Model

```
from sklearn.linear_model import LinearRegression

# Instantiate the Linear Regression model
regressor = LinearRegression()

# Train (fit) the model on the training data
regressor.fit(X_train, y_train)
```

What this cell does:

- LinearRegression(): Creates a Linear Regression model instance. No hyperparameters are needed for simple linear regression.
- regressor.fit(X_train, y_train): Trains the model by computing the OLS coefficients (b0 and b1) that minimize the sum of squared residuals between predicted and actual salaries in the training data.

Cell 3 — Predicting Test Set Results

```
# Generate predictions for the test set
y_pred = regressor.predict(X_test)

# Compare actual vs predicted side by side
comparison = np.concatenate((
    y_pred.reshape(len(y_pred), 1),
    y_test.reshape(len(y_test), 1)
), axis=1)
print(comparison)
```

What this cell does:

- regressor.predict(X_test): Applies the learned linear equation (y = b0 + b1*x) to the test features to produce salary predictions.

- np.concatenate(...): Stacks the predicted values and actual values side-by-side into a 2D array so you can visually compare them row by row. Both arrays are reshaped from 1D to 2D column vectors before concatenation.

Cell 4 — Visualisation

```
# Visualise training set results
plt.scatter(X_train, y_train, color='red', label='Actual (Train)')
plt.plot(X_train, regressor.predict(X_train), color='blue', label='Predicted Line')
plt.title('Salary vs Experience (Training Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()

# Visualise test set results
plt.scatter(X_test, y_test, color='red', label='Actual (Test)')
plt.plot(X_train, regressor.predict(X_train), color='blue', label='Predicted Line')
plt.title('Salary vs Experience (Test Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()
```

What this cell does: Creates two scatter plots. Red dots represent actual salary data points. The blue line is the linear regression best-fit line (same line in both plots, because the regression equation doesn't change). The first plot shows fit quality on training data; the second shows generalization to unseen test data. A good model will have the test data points clustered tightly around the blue line.

Notebook: multiple_linear_regression.ipynb

Dataset: 50_Startups.csv — R&D Spend, Administration, Marketing Spend, State → Profit

Key Difference: Multiple Features

```
# After standard preprocessing (including OneHotEncoding for State column):

from sklearn.linear_model import LinearRegression

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

# Display actual vs predicted profits side by side
np.set_printoptions(precision=2)
print(np.concatenate((
    y_pred.reshape(len(y_pred), 1),
    y_test.reshape(len(y_test), 1)
), axis=1))
```

What this cell does: The workflow is identical to Simple Linear Regression! This is one of scikit-learn's great strengths — the API is consistent across algorithms. The only difference is that X_train now has multiple feature columns instead of one. The LinearRegression class automatically finds the best coefficients for all features simultaneously using OLS. The output is a 2-column array showing predicted profit vs. actual profit for each test startup.

Notebook: svr.ipynb

SVR requires special handling — feature scaling on both X and y.

SVR-Specific Cells

```
from sklearn.preprocessing import StandardScaler

# SVR requires scaling on BOTH X and y
sc_X = StandardScaler()
sc_y = StandardScaler()

X_scaled = sc_X.fit_transform(X)
y_scaled = sc_y.fit_transform(y.reshape(-1, 1))

from sklearn.svm import SVR

# kernel='rbf' enables non-linear regression via the RBF kernel
regressor = SVR(kernel='rbf')
regressor.fit(X_scaled, y_scaled.ravel())

# Predict and inverse-transform to get predictions in original scale
y_pred_scaled = regressor.predict(sc_X.transform([[6.5]]))
y_pred_original = sc_y.inverse_transform(y_pred_scaled.reshape(-1, 1))
print('Predicted salary:', y_pred_original)
```

What this cell does:

- Two separate StandardScaler objects are created — one for X and one for y. SVR is distance-based (uses kernel functions), so both features and the target must be scaled to the same range to prevent numerical instability.
- y.reshape(-1, 1): StandardScaler requires a 2D input, but y is 1D. reshape(-1, 1) converts it to a column vector of shape (n, 1).
- SVR(kernel='rbf'): Creates an SVR model with the Radial Basis Function (RBF) kernel. The RBF kernel maps data into an infinite-dimensional feature space, enabling SVR to capture complex non-linear relationships.
- sc_y.inverse_transform(...): Since y was scaled before training, predictions come out in scaled form. We use the y scaler's inverse_transform to convert predictions back to the original salary values.

Chapter 3: Classification

Classification is a supervised learning task where the goal is to predict which category (class) a new observation belongs to, based on labeled training data. Unlike regression (which predicts continuous numbers), classification predicts discrete labels.

Examples include: spam detection (spam vs. not spam), disease diagnosis (cancer vs. benign), customer churn prediction (will leave vs. will stay), and image recognition (cat vs. dog vs. car).

SECTION 1 **Theory — Classification Algorithms**

3.1 Logistic Regression

Despite its name, Logistic Regression is a classification algorithm, not a regression one. It models the probability that an observation belongs to a particular class, and uses a threshold (typically 0.5) to assign the final class label.

The core of logistic regression is the Sigmoid function (also called the logistic function):

```
P(y=1) = 1 / (1 + e^(-z))    where z = b0 + b1*x1 + b2*x2 + ...
```

The sigmoid function maps any real number to a value between 0 and 1, making it interpretable as a probability. If P(y=1) >= 0.5, the model predicts class 1; otherwise, class 0.

3.2 K-Nearest Neighbours (KNN)

KNN is an intuitive, non-parametric algorithm. To classify a new data point, KNN looks at the K closest training points (neighbours) and takes a majority vote among their classes.

KNN Algorithm Steps

17. Choose k (the number of neighbours to consider). Common default: k=5.
18. Calculate the distance between the new point and every training point using a distance metric (Euclidean, Manhattan, or Minkowski).
19. Find the k training points with the smallest distances (nearest neighbours).
20. Assign the class that appears most frequently among the k neighbours (majority vote).

KNN requires feature scaling (since it is distance-based). A large-scale feature like Salary dominates distance computations over a small-scale feature like Age if not scaled.

3.3 Support Vector Machine (SVM)

SVM finds the optimal hyperplane that separates data points of different classes with the maximum possible margin. The hyperplane is the decision boundary; the margin is the distance between the hyperplane and the nearest data points of each class (called support vectors).

Maximizing the margin makes the decision boundary as robust as possible — small perturbations in the data are less likely to cause misclassifications.

3.4 Kernel SVM

Standard SVM works perfectly for linearly separable data. But real-world data is often not linearly separable — no single straight line (or flat hyperplane) can separate the classes.

Kernel SVM solves this by using the kernel trick: it maps the original feature space into a higher-dimensional space where the data becomes linearly separable, without actually computing the coordinates in that higher-dimensional space (which would be computationally expensive).

Types of Kernels

Kernel Type	Description & Use Case
Linear	For linearly separable data. Fastest — no transformation.
Polynomial	Maps data using polynomial functions. Good for structured/ordered features.
RBF (Radial Basis Function)	The most popular kernel. Maps data into infinite dimensions. Works well for most classification problems.
Sigmoid	Similar to neural network activation. Less commonly used.

3.5 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem:

```
P(A|B) = P(B|A) * P(A) / P(B)
```

In plain terms: the probability of class A given observed evidence B equals the probability of observing B given class A, times the prior probability of A, divided by the overall probability of B.

Why 'Naive'? The algorithm naively assumes that all features are statistically independent of each other, given the class label. In practice, features are often correlated, but Naive Bayes still performs remarkably well despite this simplification — especially in text classification (spam detection, sentiment analysis).

Naive Bayes Algorithm

21. Calculate the prior probability of each class: P(class) = count(class) / total_samples.
22. For each feature and each class, compute the likelihood: P(feature value | class).

23. For a new data point, compute the posterior probability for each class using Bayes' theorem.
24. Assign the class with the highest posterior probability.

3.6 Decision Tree Classification

Decision Tree Classification recursively partitions the feature space by asking binary questions (is Age > 35? Is Salary > 50,000?) and routes data points through the tree until they reach a leaf node, which represents a class label.

The algorithm chooses the splitting feature and threshold at each node to maximize Information Gain (reduce entropy/impurity). Entropy measures the disorder in a set of labels — a perfectly pure node (all one class) has entropy of 0.

3.7 Random Forest Classification

Random Forest Classification applies the same ensemble idea as Random Forest Regression, but for classification:

25. Build N decision trees, each on a different bootstrap sample of the training data.
26. At each split, consider only a random subset of features (not all features).
27. For prediction: take a majority vote across all N trees.

The key advantage: individual decision trees overfit easily. Random forests reduce this overfitting dramatically by averaging the errors across many diverse trees.

3.8 Evaluating Classification Models

The Confusion Matrix

Term	Definition
True Positive (TP)	Correctly predicted positive class (e.g., predicted 'will buy', actually did buy)
True Negative (TN)	Correctly predicted negative class (predicted 'won't buy', actually didn't)
False Positive (FP)	Incorrectly predicted positive (predicted 'will buy', actually didn't) — Type I Error
False Negative (FN)	Incorrectly predicted negative (predicted 'won't buy', actually did) — Type II Error

Metric	Formula & Interpretation

Accuracy	(TP + TN) / Total — Overall correct predictions rate
Precision	TP / (TP + FP) — Of all predicted positives, how many were actually positive?
Recall (Sensitivity)	TP / (TP + FN) — Of all actual positives, how many did the model catch?
F1 Score	2 * (Precision * Recall) / (Precision + Recall) — Harmonic mean, best for imbalanced datasets

SECTION 2 Practical — Classification Notebooks

Notebook: logistic_regression.ipynb

Dataset: Social_Network_Ads.csv — Age, Estimated Salary → Purchased (0 or 1)

Complete Implementation

```
# Step 1: Data Preprocessing
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

dataset = pd.read_csv('Social_Network_Ads.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test  = sc.transform(X_test)

# Step 2: Train Logistic Regression
from sklearn.linear_model import LogisticRegression

classifier = LogisticRegression(random_state=0)
classifier.fit(X_train, y_train)

# Step 3: Predict a single new observation
print(classifier.predict(sc.transform([[30, 87000]])))

# Step 4: Predict the test set
y_pred = classifier.predict(X_test)

# Step 5: Evaluate with Confusion Matrix and Accuracy
```

```
from sklearn.metrics import confusion_matrix, accuracy_score

cm = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:\n', cm)
print('Accuracy:', accuracy_score(y_test, y_pred))
```

Cell-by-cell explanation:

- Data Preprocessing: Standard pipeline — load CSV, create X and y, split 75/25 (smaller datasets often use a larger test portion), scale features. Both Age and Salary must be scaled because Logistic Regression is distance/gradient-based.
- LogisticRegression(random_state=0): Instantiates the classifier. random_state ensures reproducible results since the solver uses some randomness in initialization.
- classifier.fit(X_train, y_train): Trains the model using Maximum Likelihood Estimation to find the optimal weights that maximize the probability of observing the training labels.
- classifier.predict(sc.transform([[30, 87000]])): Predicts whether a 30-year-old with a salary of 87,000 will purchase. IMPORTANT: the new data point must be wrapped in double brackets (to make it 2D) and must be scaled using the same scaler fitted on training data.
- confusion_matrix(y_test, y_pred): Returns a 2x2 matrix [[TN, FP], [FN, TP]] showing the counts of each prediction type.
- accuracy_score(y_test, y_pred): Returns the proportion of correct predictions (TP + TN) / total.

Notebook: knn.ipynb — Key Differences

```
from sklearn.neighbors import KNeighborsClassifier

# n_neighbors=5: Consider 5 nearest neighbours for voting
# metric='minkowski', p=2: Equivalent to Euclidean distance
classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
classifier.fit(X_train, y_train)
```

What this cell does: KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2) creates a KNN classifier. n_neighbors=5 means the 5 closest training points vote on the class of each new point. metric='minkowski' with p=2 is the Minkowski distance formula with p=2, which is exactly the Euclidean distance. The rest of the pipeline (predict, confusion matrix, accuracy) is identical to Logistic Regression.

Notebook: random_forest_classification.ipynb — Key Differences

```
from sklearn.ensemble import RandomForestClassifier

# n_estimators=10: Build 10 decision trees
# criterion='entropy': Use Information Gain (entropy) to select splits
classifier = RandomForestClassifier(
    n_estimators=10,
    criterion='entropy',
    random_state=0
```

```
)
classifier.fit(X_train, y_train)
```

What this cell does: RandomForestClassifier(n_estimators=10, criterion='entropy') creates an ensemble of 10 decision trees. Each tree uses entropy (information gain) as its split criterion. The random_state=0 ensures that both the bootstrap sampling and feature selection randomness are reproducible. The final prediction is the majority vote across all 10 trees.

Chapter 4: Clustering

Clustering is an unsupervised learning technique. Unlike supervised learning (regression and classification), clustering algorithms have no labeled training data — there is no target variable y to learn from. Instead, clustering algorithms discover natural groupings (clusters) within the data based purely on the similarity or distance between data points.

Real-world applications include: customer segmentation (group customers by purchasing behaviour), document categorization, anomaly detection, gene expression analysis, and image compression.

SECTION 1 Theory — Clustering Algorithms

4.1 K-Means Clustering

K-Means is the most popular and widely used clustering algorithm. Given a target number of clusters K (specified by the user), K-Means partitions the data into K groups such that points within the same group are as similar as possible.

K-Means Algorithm

28. Choose K — the number of clusters to form.
29. Initialize K centroids by randomly placing them in the feature space.
30. Assignment Step: Assign each data point to the nearest centroid (based on Euclidean distance). This forms K clusters.
31. Update Step: Recalculate each centroid as the mean of all points currently assigned to it.
32. Repeat steps 3 and 4 until the centroids no longer move significantly (convergence).

Random Initialization Trap & K-Means++

Standard K-Means randomly initializes centroids, which can lead to poor clustering if the initial positions are unlucky. Different runs can produce completely different results. K-Means++ solves this by initializing centroids in a smarter way: the first centroid is chosen randomly, and each subsequent centroid is chosen with probability proportional to its squared distance from the nearest already-chosen centroid. This encourages centroids to be spread out initially, leading to better and more consistent results. scikit-learn uses K-Means++ by default.

The Elbow Method — Choosing Optimal K

How do you decide how many clusters (K) to use? The Elbow Method plots the Within Cluster Sum of Squares (WCSS) against different values of K. WCSS measures the total variance within each cluster — lower WCSS means tighter clusters.

As K increases, WCSS always decreases (more clusters means smaller groups with less internal variance). The Elbow Method looks for the 'elbow' — the point where adding more clusters gives diminishing returns. This is the optimal K.

4.2 Hierarchical Clustering

Hierarchical Clustering builds a tree of clusters (called a dendrogram) without requiring a pre-specified number of clusters K. The two main types are:

Agglomerative (Bottom-Up)	Divisive (Top-Down)
Start: each point is its own cluster	Start: all points form one big cluster
Merge closest pairs of clusters step by step	Recursively split clusters into smaller ones
Stop: when all points are in one cluster	Stop: when each point is its own cluster
More commonly used in practice	Computationally expensive — rarely used

Dendrograms

A dendrogram is a tree diagram that shows the hierarchical structure of merges. The y-axis represents the distance (dissimilarity) at which clusters were merged. To determine the optimal number of clusters, look for the longest vertical line that no horizontal line crosses — the number of vertical lines at that level gives the optimal K.

Linkage Methods

When merging clusters, there are multiple ways to measure the distance between two clusters:

- Single Linkage: Distance = minimum distance between any two points in the two clusters. Produces elongated, chain-like clusters.
- Complete Linkage: Distance = maximum distance between any two points. Produces compact, spherical clusters.
- Average Linkage: Distance = average of all pairwise distances between points in the two clusters.
- Ward Linkage (default in scikit-learn): Minimizes the total within-cluster variance at each merge step. Generally produces the most balanced clusters.

SECTION 2 Practical — Clustering Notebooks

Notebook: kmeans.ipynb

Dataset: Mall_Customers.csv — Annual Income, Spending Score

Cell 1 — Finding Optimal K with the Elbow Method

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import KMeans

dataset = pd.read_csv('Mall_Customers.csv')

# Use Annual Income and Spending Score columns
X = dataset.iloc[:, [3, 4]].values

# Test K from 1 to 10 and record WCSS for each
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++',
                    max_iter=300, n_init=10, random_state=0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)   # inertia_ stores WCSS

# Plot the Elbow Curve
plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

What this cell does:

- dataset.iloc[:, [3, 4]].values: Selects only the Annual Income and Spending Score columns (indices 3 and 4). These are the two features we cluster on.
- KMeans(n_clusters=i, init='k-means++'): The init='k-means++' ensures smart centroid initialization. max_iter=300 sets the maximum number of iterations for convergence. n_init=10 runs the algorithm 10 times with different random initializations and keeps the best result.
- kmeans.inertia_: The .inertia_ attribute stores the WCSS value for the fitted model. We append this to the wcss list.
- The resulting Elbow Plot shows WCSS decreasing as K increases. Look for the 'elbow' bend — in this dataset, K=5 is typically optimal.

Cell 2 — Training the Final K-Means Model

```
# Train with optimal K=5 (from the elbow method)
kmeans = KMeans(n_clusters=5, init='k-means++',
                max_iter=300, n_init=10, random_state=0)
y_kmeans = kmeans.fit_predict(X)
```

What this cell does: fit_predict(X) both trains the K-Means model and returns the cluster label for each data point in a single step. y_kmeans is an array of integers (0 through 4) indicating which cluster each customer belongs to.

Cell 3 — Visualising the Clusters

```
# Plot each cluster in a different color
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1],
            s=100, c='red', label='Cluster 1')
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1],
            s=100, c='blue', label='Cluster 2')
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1],
            s=100, c='green', label='Cluster 3')
plt.scatter(X[y_kmeans == 3, 0], X[y_kmeans == 3, 1],
            s=100, c='cyan', label='Cluster 4')
plt.scatter(X[y_kmeans == 4, 0], X[y_kmeans == 4, 1],
            s=100, c='magenta', label='Cluster 5')

# Plot cluster centroids
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            s=300, c='yellow', label='Centroids', marker='*')

plt.title('Customer Segments')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```

What this cell does: Plots each cluster in a distinct color using boolean indexing (X[y_kmeans == 0, 0] selects the Annual Income values of all points in Cluster 0). The cluster centroids are plotted as large yellow stars using kmeans.cluster_centers_. This plot reveals distinct customer segments: e.g., high income & high spending ('target customers'), high income & low spending ('careful'), low income & high spending ('impulsive'), etc.

Notebook: hierarchical_clustering.ipynb — Key Cells

```
import scipy.cluster.hierarchy as sch

# Step 1: Plot dendrogram to find optimal number of clusters
dendrogram = sch.dendrogram(sch.linkage(X, method='ward'))
plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distances')
plt.show()

# Step 2: Train Agglomerative Clustering with optimal K (e.g., 5)
from sklearn.cluster import AgglomerativeClustering

hc = AgglomerativeClustering(n_clusters=5,
                             affinity='euclidean',
                             linkage='ward')
y_hc = hc.fit_predict(X)
```

Cell-by-cell explanation:

- sch.linkage(X, method='ward'): Computes the linkage matrix — a hierarchical merge history. Ward's method minimizes within-cluster variance at each merge.

- sch.dendrogram(...): Plots the hierarchical tree. To find the optimal K, look for the longest vertical line that isn't crossed by any horizontal line — cutting the dendrogram there gives the natural cluster structure.
- AgglomerativeClustering(n_clusters=5, affinity='euclidean', linkage='ward'): Trains the agglomerative clustering model with 5 clusters. affinity='euclidean' uses Euclidean distance to measure point similarity. linkage='ward' matches what was used in the dendrogram.

Chapter 5: Deep Learning

Deep learning is a subset of machine learning inspired by the structure and function of the human brain. It uses Artificial Neural Networks (ANNs) with many layers to learn hierarchical representations of data. The key enablers of deep learning's recent success are large amounts of data and significant computational power (especially GPUs).

SECTION 1 Theory — Neural Networks

5.1 Artificial Neural Networks (ANNs)

Biological Inspiration

The human brain contains approximately 86 billion neurons interconnected via synapses. Each neuron receives electrical signals through its dendrites, processes them, and transmits a signal through its axon to the next neuron. Deep learning mimics this structure computationally.

ANN Structure

Component	Description
Input Layer	Receives the raw input features (independent variables). One node per feature.
Hidden Layers	Intermediate layers that learn abstract representations. 'Deep' networks have many hidden layers.
Output Layer	Produces the final prediction. One node for binary classification; multiple nodes for multi-class.
Weights (W)	Learnable parameters on each connection between neurons. Initialized randomly, optimized during training.
Bias (b)	An additional learnable parameter that shifts the activation function.
Activation Function	Applied at each neuron to introduce non-linearity, enabling the network to learn complex patterns.

Activation Functions

Activation Function	When to Use

Threshold (Step)	Binary output: fire (1) or don't fire (0). Rarely used in modern networks.
Sigmoid	Binary classification output layer. Outputs probability between 0 and 1.
ReLU (Rectified Linear Unit)	Hidden layers in deep networks. Fastest and most commonly used. Avoids vanishing gradient.
Softmax	Multi-class classification output layer. Outputs a probability distribution across all classes.
TanH (Hyperbolic Tangent)	Hidden layers when zero-centered outputs are needed. Range: (-1, 1).

5.2 Training Neural Networks

Forward Propagation

During forward propagation, inputs flow through the network layer by layer, with each neuron applying its weights, bias, and activation function to compute its output. The final layer produces the prediction y_hat.

Cost Function

The cost (or loss) function measures how far the prediction y_hat is from the true label y. For binary classification, Binary Cross-Entropy is used. For regression, Mean Squared Error (MSE) is common.

Backpropagation

Backpropagation computes the gradient of the cost function with respect to each weight in the network using the chain rule of calculus. These gradients tell us how much each weight contributed to the error.

Gradient Descent and Its Variants

Variant	Description
Batch Gradient Descent	Computes gradients using the entire dataset. Very slow for large datasets. Stable convergence.
Stochastic Gradient Descent (SGD)	Updates weights after each single data point. Very fast but noisy (erratic convergence).
Mini-Batch Gradient Descent	Updates weights after each small batch (e.g., 32 samples). Best of both worlds — the most commonly used approach in practice.

5.3 Convolutional Neural Networks (CNNs)

CNNs are specialized neural networks designed for processing grid-structured data — particularly images. They automatically learn spatial hierarchies of features (edges → shapes → objects) through convolution operations.

CNN Architecture Steps

33. Convolution: Apply feature detectors (filters/kernels) to the input image. Each filter slides across the image, computing a dot product at each position, producing a Feature Map that highlights specific patterns (e.g., edges, curves).
34. ReLU Activation: Applied element-wise to the feature map. Removes negative values (introduces non-linearity), making the network capable of learning complex patterns.
35. Pooling (Downsampling): Reduces the spatial dimensions of each feature map. Max Pooling selects the maximum value from each patch. This reduces computation and makes the network invariant to small translations.
36. Flattening: Converts the 2D pooled feature maps into a 1D vector. This vector is fed into the fully connected layers.
37. Full Connection (Dense Layers): Standard ANN layers that combine all learned features to produce the final classification output.

Image Type	Representation in CNN
Grayscale (B&W)	2D array of pixel values (height x width). Single channel.
Color (RGB)	3D array (height x width x 3 channels for Red, Green, Blue).

SECTION 2 Practical — Deep Learning Notebooks

Notebook: ann.ipynb

Dataset: Churn_Modelling.csv — Bank customer data → Predicting customer churn (will the customer leave?)

Cell 1 — Data Preprocessing for ANN

```
import numpy as np
import pandas as pd
import tensorflow as tf

dataset = pd.read_csv('Churn_Modelling.csv')
```

```
# Features: RowNumber, CustomerId, Surname dropped (irrelevant)
X = dataset.iloc[:, 3:-1].values
y = dataset.iloc[:, -1].values

# Encode Gender: Male=1, Female=0 (Label Encoding)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X[:, 2] = le.fit_transform(X[:, 2])

# One-Hot Encode Geography (France, Germany, Spain)
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(), [1])],
    remainder='passthrough'
)
X = np.array(ct.fit_transform(X))

# Split and Scale
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test  = sc.transform(X_test)
```

What this cell does: Prepares the dataset for an ANN. Notable steps: (1) The first three columns (RowNumber, CustomerId, Surname) are dropped with iloc[:, 3:-1] because they are identifiers with no predictive value. (2) Gender is label-encoded (binary, so no one-hot encoding needed). (3) Geography is one-hot encoded (3 countries → 3 binary columns). (4) Feature scaling is mandatory for ANNs because gradient descent converges much faster when all features are on the same scale.

Cell 2 — Building the ANN

```
# Initialize the ANN
ann = tf.keras.models.Sequential()

# First Hidden Layer: 6 neurons, ReLU activation
ann.add(tf.keras.layers.Dense(units=6, activation='relu'))

# Second Hidden Layer: 6 neurons, ReLU activation
ann.add(tf.keras.layers.Dense(units=6, activation='relu'))

# Output Layer: 1 neuron, Sigmoid activation (binary classification)
ann.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

What this cell does:

- tf.keras.models.Sequential(): Creates a linear stack of layers where data flows from one layer to the next in sequence.

- Dense(units=6, activation='relu'): Adds a fully connected hidden layer with 6 neurons and ReLU activation. ReLU is preferred for hidden layers because it avoids the vanishing gradient problem and trains significantly faster than sigmoid/tanh.
- Dense(units=1, activation='sigmoid'): The output layer has a single neuron with sigmoid activation, producing a probability between 0 and 1. If probability >= 0.5, predict churn (1); otherwise predict no churn (0).

Cell 3 — Compiling and Training the ANN

```
# Compile: specify optimizer, loss function, and evaluation metric
ann.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train for 100 epochs with batch size 32
ann.fit(X_train, y_train, batch_size=32, epochs=100)
```

What this cell does:

- optimizer='adam': Adam (Adaptive Moment Estimation) is the most widely used optimizer. It adapts the learning rate for each parameter individually, converging faster and more reliably than standard SGD.
- loss='binary_crossentropy': The appropriate loss function for binary classification. It measures the difference between predicted probabilities and true binary labels.
- batch_size=32: Updates weights after processing 32 training examples. Mini-batch training provides a good balance between computational efficiency and stability.
- epochs=100: The entire training dataset is passed through the network 100 times. The model progressively improves its weights with each epoch.

Cell 4 — Making Predictions

```
# Predict on a single new customer
# Input: [Geography, Credit Score, Gender, Age, Tenure, Balance,
#         NumProducts, HasCrCard, IsActiveMember, EstimatedSalary]
print(ann.predict(sc.transform([[1, 0, 0, 600, 1, 40, 3, 60000,
                                 2, 1, 1, 50000]])) > 0.5)

# Predict on test set
y_pred = ann.predict(X_test)
y_pred = (y_pred > 0.5)   # Convert probabilities to binary labels

# Confusion Matrix and Accuracy
from sklearn.metrics import confusion_matrix, accuracy_score
print(confusion_matrix(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test, y_pred))
```

What this cell does:

- ann.predict(sc.transform([[...]])): Always scale new input data with the same scaler used during training. The double brackets make the input 2D (required by the model). The result is a probability between 0 and 1.
- y_pred > 0.5: Converts the probability predictions into binary class labels. Any probability above 0.5 is classified as 1 (churn), anything below as 0 (stay).

Chapter 6: Additional Topics

6.1 Association Rule Learning

Association Rule Learning discovers interesting relationships (rules) between variables in large datasets. Classic use case: Market Basket Analysis — 'Customers who buy X also tend to buy Y.'

Key Metrics

Metric	Definition
Support	How frequently items appear together in the dataset: count(X and Y) / total transactions
Confidence	Given X was purchased, probability Y was also purchased: count(X and Y) / count(X)
Lift	How much more likely X and Y are purchased together vs. independently: Confidence / Support(Y)

Apriori Algorithm

The Apriori algorithm uses a 'bottom-up' approach: it first identifies individual items meeting a minimum support threshold, then progressively builds larger itemsets by combining smaller frequent ones. This avoids evaluating every possible itemset combination.

ECLAT Algorithm

ECLAT (Equivalence Class Transformation) is a simpler alternative that only computes Support (not Confidence or Lift). It uses a vertical data format (item → list of transactions containing it) which makes intersection computations faster. Use ECLAT when you only need to identify frequently co-occurring items.

6.2 Reinforcement Learning

Reinforcement Learning (RL) is a paradigm where an agent learns to make decisions by interacting with an environment, receiving rewards for good actions and penalties for bad ones. Unlike supervised learning, there is no labeled training data — the agent learns from experience.

Classic problem: The Multi-Armed Bandit Problem. Imagine K slot machines, each with an unknown reward distribution. The goal is to maximize total reward over time by intelligently

choosing which machine to play, balancing exploration (trying unknown machines) and exploitation (playing the best-known machine).

UCB (Upper Confidence Bound)	Thompson Sampling
Deterministic algorithm	Probabilistic (Bayesian) algorithm
Chooses the arm with highest UCB score: mean + confidence interval	Samples from a Beta distribution for each arm's estimated reward probability
Deterministic: same inputs always give same action	Stochastic: samples add natural exploration
Faster convergence in low-noise settings	More natural exploration; often outperforms UCB long-term

6.3 Natural Language Processing (NLP)

NLP is the subfield of AI focused on enabling computers to understand, interpret, and generate human language. It bridges the gap between unstructured text and structured machine-readable data.

Bag of Words Model

The Bag of Words (BoW) model is one of the simplest text representation techniques. It converts text into numerical feature vectors by counting word frequencies, completely ignoring grammar and word order.

Sentiment Analysis Pipeline

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

nltk.download('stopwords')

corpus = []
for i in range(0, len(dataset)):
    # Remove non-letter characters, lowercase
    review = re.sub('[^a-zA-Z]', ' ', dataset['Review'][i])
    review = review.lower().split()

    # Remove stopwords and apply stemming
    ps = PorterStemmer()
    all_stopwords = stopwords.words('english')
    all_stopwords.remove('not')   # 'not' is important for sentiment!
    review = [ps.stem(word) for word in review
              if word not in set(all_stopwords)]

    corpus.append(' '.join(review))

# Build the Bag of Words model (max 1500 features)
cv = CountVectorizer(max_features=1500)
```

```
X = cv.fit_transform(corpus).toarray()
y = dataset.iloc[:, -1].values
```

What this cell does:

- re.sub('[^a-zA-Z]', ' ', ...): Removes all non-letter characters (punctuation, numbers, special symbols) from the review text, replacing them with spaces.
- .lower().split(): Converts text to lowercase and splits into individual words.
- stopwords.words('english'): A list of common English words (the, is, and, ...) that carry little semantic meaning. We remove them to reduce noise. We explicitly keep 'not' because it reverses sentiment (e.g., 'not good' vs. 'good').
- PorterStemmer: Reduces words to their root form (e.g., 'loving', 'loved', 'loves' all become 'love'). This reduces vocabulary size and groups related word forms together.
- CountVectorizer(max_features=1500): Builds the Bag of Words matrix. max_features=1500 limits the vocabulary to the 1,500 most frequent words, preventing an excessively large feature matrix. The result X is a sparse matrix of word counts.

6.4 Dimensionality Reduction

As datasets grow in the number of features, the 'curse of dimensionality' makes ML models increasingly difficult to train and interpret. Dimensionality reduction techniques address this by reducing the number of features while preserving the most important information.

Technique	Type & Description
PCA (Principal Component Analysis)	Unsupervised. Finds the directions of maximum variance (principal components) and projects data onto them. Reduces features while preserving overall variance.
LDA (Linear Discriminant Analysis)	Supervised. Maximizes class separability. Projects data onto axes that best separate different classes. Used before classification.
Kernel PCA	Non-linear PCA. Uses the kernel trick to handle non-linearly separable data. Best when PCA's linear projection is insufficient.

Implementing PCA

```
from sklearn.decomposition import PCA

# Reduce to 2 components (for visualization)
pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train)
X_test_pca  = pca.transform(X_test)

# Check explained variance ratio
print('Explained Variance Ratio:', pca.explained_variance_ratio_)
print('Total Variance Explained: {:.1f}%'.format(
    sum(pca.explained_variance_ratio_) * 100))
```

What this cell does:

- PCA(n_components=2): Reduces the feature space to 2 principal components. Choose n_components to retain at least 95% of total variance (check explained_variance_ratio_).
- pca.fit_transform(X_train): Fits PCA on training data — finds the principal components — and transforms the training features into the new 2D space.
- pca.transform(X_test): Applies the SAME transformation (learned from training) to the test data. Do not fit on test data.
- explained_variance_ratio_: Shows what proportion of total variance each principal component captures. This guides you in choosing the right number of components.

6.5 XGBoost

XGBoost (Extreme Gradient Boosting) is one of the most powerful and widely used machine learning algorithms in production and competitions. It is an ensemble method based on Gradient Boosting, building trees sequentially where each new tree corrects the errors of its predecessor.

Key Advantages of XGBoost

- Speed: Highly optimized C++ implementation with parallel processing. Much faster than standard gradient boosting.
- Regularization: Built-in L1 (Lasso) and L2 (Ridge) regularization prevents overfitting.
- Handles Missing Values: XGBoost can handle NaN values internally.
- Flexibility: Works for regression, binary classification, and multi-class classification.

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_val_score

# Train XGBoost
classifier = XGBClassifier()
classifier.fit(X_train, y_train)

# Evaluate on test set
y_pred = classifier.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))

# K-Fold Cross Validation (k=10) for robust evaluation
accuracies = cross_val_score(
    estimator=classifier,
    X=X_train,
    y=y_train,
    cv=10
)
print(f'Mean Accuracy: {accuracies.mean():.2%}')
print(f'Std Deviation: {accuracies.std():.2%}')
```

What this cell does:

- XGBClassifier(): Creates an XGBoost classification model with default hyperparameters (which are already well-tuned for many problems).
- cross_val_score(cv=10): Performs 10-Fold Cross Validation — splits the training data into 10 equal parts, trains on 9 and validates on 1, cycling through all 10 combinations. The mean accuracy gives a more reliable performance estimate than a single train/test split.
- accuracies.mean() and accuracies.std(): The mean accuracy represents expected real-world performance. The standard deviation measures variance — a low std (< 1%) means the model is stable and consistent across different data splits.

End of Machine Learning A-Z Complete Reference Guide

Keep learning. Keep building. Keep shipping.